

5. Chat / Messaging (WhatsApp / Slack)

Interviewer: Design the backend for a chat app like WhatsApp or Slack — 1:1 and group messaging, delivered in real time, with delivery/read receipts and online presence. Assume hundreds of millions of users.

Candidate: This is a fun one because the hard part isn't storage or reads — it's that the **server has to push data to a client it's holding a connection to**, at massive scale, and still guarantee the message isn't lost if that connection is gone. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before I size anything: 1. 1:1 and group chat both — how big do groups get? Family-chat-sized (dozens) or Slack-workspace-sized (thousands)? 2. Do we need delivery receipts (sent/delivered) *and* read receipts, or just one? 3. Presence — online/offline only, or also "typing..." indicators? 4. If the recipient is offline, what happens — do we just hold the message, or also push a mobile notification? 5. Is end-to-end encryption in scope, or can the server see plaintext?

Interviewer: Groups up to a few thousand members. Both delivery and read receipts. Online/offline presence plus "typing." If offline, the message must be waiting for them when they come back, *and* fire a push notification. E2E encryption — just touch on it briefly at the end, don't design the crypto.

Candidate: Good, that pins down the shape. Requirements:

Functional - Send/receive 1:1 and group messages; message shows up live if the recipient is connected. - Delivery receipts (sent → delivered → read) and typing/online presence. - Message history — a new device or a reopened chat can page back through history. - Offline recipients get the message queued (store-and-forward) + a push notification.

Non-functional — this is where the real problem lives - **Real-time:** sub-200ms delivery when both users are online. - **Massive persistent concurrency:** tens to hundreds of millions of clients each holding an *open* connection, waiting to be pushed to — not polling. - **Ordering:** messages in one conversation must render in a consistent order, even though senders and servers are physically distributed. -

Durability: a message is written once it's accepted; it must never silently vanish, even if the recipient is offline for a week. - **At-least-once delivery with dedup** — a retried send must not become two visible messages.

2. Estimation — why this isn't a "database" problem

Candidate: Let me size two very different things: the *message* volume, and the *connection* count — because the second one is what makes this hard.

```
500M DAU, ~50 messages/user/day sent (not received — fan-out multiplies receive-side).
messages/day = 500M * 50 = 25B/day
writes/sec   = 25B / 86,400 ≈ 290k/sec (peak 2-3x → ~700k-900k/sec)
```

Concurrent OPEN CONNECTIONS (the real bottleneck):

~500M DAU, say 20% concurrently online at peak → ~100M sockets held open, simultaneously, all day, doing nothing most of the time but ready to receive a push.

One event-loop server (epoll/kqueue) can hold ~50k-100k idle WebSocket connections before running out of memory/file descriptors.

→ 100M / 75k ≈ ~1,300+ gateway servers JUST to hold sockets open, before a single message is even routed.

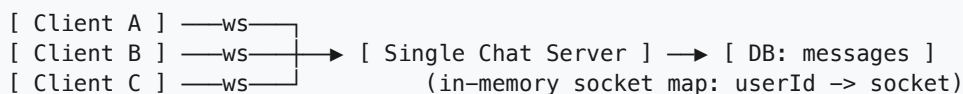
Storage: 25B msgs/day * ~200 bytes = ~5 TB/day → ~1.8 PB/year (pre-replication).

⇒ The write throughput is high but very solvable (Kafka/Cassandra eat this for breakfast). The NEW problem this system has that others don't: given 100M idle-but-open sockets spread across 1,300+ servers, how do I find "which server is holding user B's socket" and forward a message to it in <200ms?

Candidate: That routing question — a connection registry — is the spine of this whole design. Everything else (storage, ordering, receipts) hangs off it.

3. V1 — a single chat server (and why it can't survive)

Candidate: Simplest possible version to agree on terms: one server holds every socket.



A sends a message → the server looks up B's socket **in its own memory** → writes to it directly. Trivial, and even *correct* for ordering, since one process makes every decision.

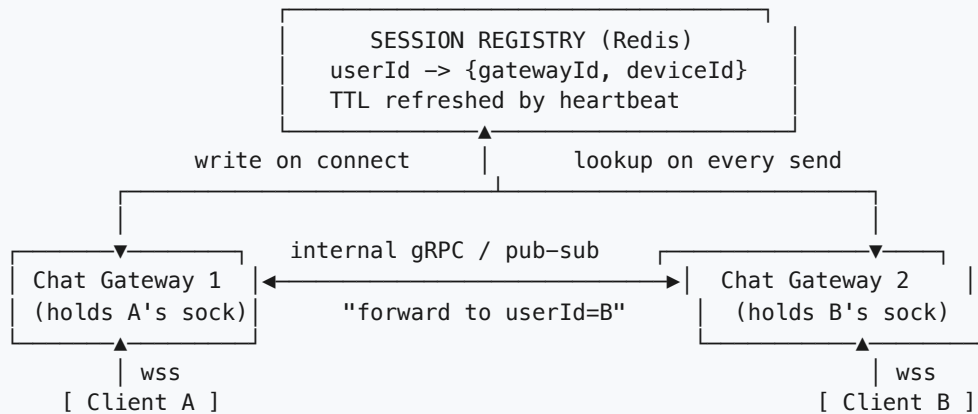
Why it dies: one process caps out around ~50k-100k sockets. We need 100M concurrent connections — there's no vertical scale path past that, so we need **many gateway servers**, which immediately creates the problem V1 didn't have: *A's socket and B's socket are probably on two different machines that don't share memory.*

4. The key idea — a connection gateway layer + a session registry

Interviewer: Right, so now split it across many servers. How does A's message find B if they're connected to different machines?

Candidate: This is the crux. I split the role in two:

- **Chat gateway servers** — stateless about *message logic*, but each one is **stateful about which sockets it physically holds** (an in-memory map of `userId -> local socket`). Hundreds to thousands of these, horizontally scaled, each capped at ~50k-100k connections.
- **A session registry (Redis)** — the *global* map of `userId -> gatewayId (+ deviceId)`. When a client connects, its gateway writes `session:{userId} = {gatewayId, connectedAt}` with a TTL refreshed by heartbeats. This is the piece that lets *any* gateway find *any* user.

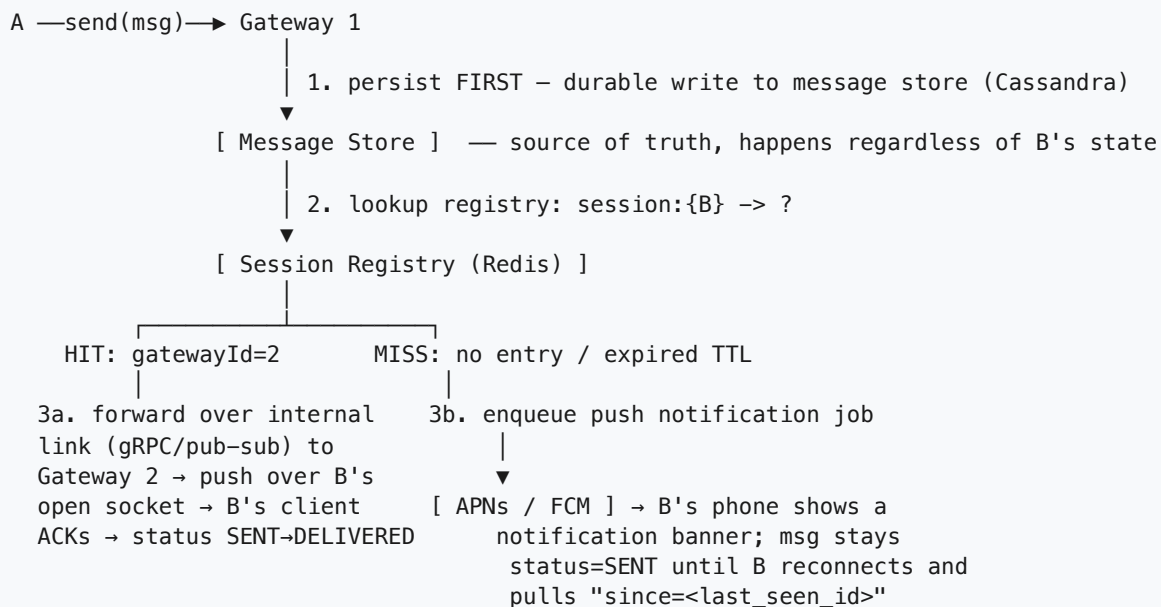


Why Redis for the registry, specifically: in-memory KV means a sub-millisecond lookup, and it's naturally shardable by `userId` — no single hot node. TTL gives free "did this session go stale" detection: if a gateway crashes without a clean disconnect, the key expires and the user is treated as offline until they reconnect. **Why not a database table:** the registry is rewritten on every connect/heartbeat (soft, volatile state) — hammering Postgres with that write rate for data we're fine losing on restart is the wrong trade.

5. The send path — online delivery and offline store-and-forward

Interviewer: Walk me through one message end to end, A to B, different gateways.

Candidate: Two branches off the same first step.



The one rule that matters: persist before you push. Step 1 happens unconditionally, before we even know if B is reachable. The socket layer is best-effort and can drop at any instant; the store is what makes the message durable — it is the offline mailbox, so "B is offline" and "B is online but the forward failed" both degrade to the exact same safe fallback: nothing is lost, delivery is just delayed and a push notification wakes B up. Coming back online is a normal history read filtered to "since my last cursor," nothing bespoke.

Internal fan-out transport — gRPC direct call vs. pub/sub: for a 1:1 message, Gateway 1 can call Gateway 2 directly (gRPC, address known from the registry). For **group messages**, direct calls to potentially thousands of gateways don't scale — instead publish once to a pub/sub layer (Redis Pub/Sub or a Kafka topic keyed by conversation) that every relevant gateway subscribes to, so one publish reaches every gateway holding a member's socket without the sender's gateway needing to enumerate them.

6. Scaling to hundreds of millions of connections

Interviewer: Now widen it — hundreds of millions of concurrent sockets, not thousands. What breaks first and how do you fix it?

Candidate: Three things, in order of how fast they'd hurt:

1. **Gateway fleet size.** At ~75k sockets/server and 100M+ concurrent connections, that's 1,300+ gateway boxes idling most of the time — fine, since gateways are cheap, stateless-to-deploy, and horizontally autoscaled; RAM for socket buffers is the constraint, not CPU.
2. **Registry lookup rate.** Every message does at least one registry read — at 900k msgs/sec that's 900k+ Redis GETs/sec, solved by sharding the registry cluster by `userId` hash so no one shard absorbs it all, same pattern as the burger-giveaway sharded counters.
3. **Load balancer stickiness.** The initial `GET /ws` (HTTP Upgrade) must be routed by an L7 load balancer that understands the Upgrade header; once upgraded, that connection is **pinned to one**

gateway for its entire lifetime — no per-request rebalancing anymore. **Connection draining** matters: retiring a gateway during a deploy stops it accepting *new* connections and sends existing clients a "please reconnect" signal so they re-handshake elsewhere, instead of just killing 75k sockets at once.

```
Client
|
[ L7 LB ] -- TLS termination, recognizes Upgrade: websocket,
|          picks a gateway ONCE per connection, then gets out of the way
|          → Chat Gateway pool (1,300+ nodes, each ~75k long-lived sockets)
|          → REST services (history, contacts, auth) -- stateless, ordinary LB
```

7. Multi-region

Interviewer: Now go global — a user in São Paulo messaging a user in Tokyo. How does that change things?

Candidate: Two separate concerns: where the client connects, and where the message flows.

- **Connection locality:** GeoDNS/anycast routes each client to the *nearest* regional gateway pool — that's purely about minimizing the socket's round-trip latency for heartbeats and pushes, nothing to do with who they're chatting with.
- **Cross-region message flow:** the sender's local gateway persists to its **local** message store region first (fast, no cross-ocean wait on the write), and *in parallel* forwards the live message directly gateway-to-gateway across regions to the recipient if they're connected there — we do **not** wait for cross-region storage replication before pushing. Storage replicates asynchronously in the background so scroll-back history is a local read in every region a conversation's members are in.

```
[US: Client A] --ws--> [US Gateway] --persist--> [US message store]
                                     |
                                     | async replicate
                                     v
                                     [EU message store]
                                     |
                                     v
                                [EU Gateway] --ws--> [EU: Client B]
```

If the transatlantic link is down, local delivery within each region keeps working; cross-region messages queue at the edge and flush once the link heals — never blocking or losing the already-persisted local copy.

8. Latency — where the milliseconds go

Interviewer: You promised <200ms online-to-online. Break down the budget.

Candidate:

A -> Gateway 1 (wss, same region)	~10-20 ms
Gateway 1: durable write (Cassandra, quorum, local DC)	~5-15 ms
Gateway 1: registry lookup (Redis, in-memory)	~1 ms
Gateway 1 -> Gateway 2 (internal, same DC gRPC/pubsub)	~2-5 ms
Gateway 2 -> B (wss push)	~10-20 ms

total, same region	~30-60 ms (comfortably < 200ms)
cross-region (add one WAN hop each way)	+100-150 ms -> still workable

Where I'd cut further if needed: run the persist and the forward **in parallel** instead of sequentially — Gateway 1 only needs the write *durably queued* (a fast local quorum) before it kicks off the registry lookup and forward, not a full round-trip wait. What must stay ordered: the write is durable *before* we tell A "sent"; the forward is best-effort and races the persist.

9. Consistency and ordering — the real correctness question here

Interviewer: Two people in a group send messages within the same millisecond, hitting two different gateways. How do you guarantee everyone sees a consistent order?

Candidate: This is the trickiest correctness question in the whole design, more than scarcity-counting problems, because there's no single lock to serialize through — messages arrive at *different* gateways in *different* regions with *clock skew* between them.

- **Never trust wall-clock timestamps alone for ordering across servers.** Two gateways' clocks can disagree by tens of milliseconds; "sort by `created_at`" would flip real send order.
- **Use a time-based, monotonically-sortable ID as the ordering key** — e.g. a Cassandra `timeuuid` or a Snowflake-style ID (timestamp bits + a server/shard ID + a per-ms sequence). This gives every message a globally comparable sort key *without* a central counter, because the timestamp component dominates and the shard/sequence bits only break ties.
- **Per-sender order is trivially preserved** — a single client sends over one socket, so its own messages arrive in the order it sent them (TCP preserves order on one connection).
- **Cross-sender order in a group is "good enough," not perfectly linearized.** WhatsApp/Slack don't need a total global order the way a ledger does — they need *causally sensible* order (a reply doesn't render before the message it replies to, for the vast majority of cases). Sorting by the time-based ID at read time gives a stable, consistent-enough order for every reader, even if two near-simultaneous sends from different regions could theoretically be swapped by a few ms — no user perceives that as "wrong."
- **Delivery/read status is a monotonic state machine**, `SENT -> DELIVERED -> READ`, never backwards. A late-arriving `DELIVERED` ack must not overwrite an already-`READ` status — use a conditional write (only advance the state machine forward) rather than a blind overwrite.

What's strong vs. eventual, stated plainly: - **Strong:** per-conversation message durability (once accepted, it's written) and the monotonic receipt state machine. - **Eventual:** presence ("online" 2 seconds stale is fine), the exact interleave order of two messages from different senders arriving within milliseconds of each other, and cross-region replication of history.

10. Async — what moves off the critical path

Interviewer: What do you deliberately make asynchronous, and why?

Candidate: Anything that isn't "did the message reach the recipient's screen":

```
[ Message persisted + pushed ] <-- synchronous, critical path, <200ms
├──> push notification job (APNs/FCM)      -- async, offline case only
├──> delivery/read receipt propagation    -- async, fire-and-forget
├──> group fan-out beyond the first N members -- async queue
├──> search-index update, analytics event  -- async
├──> cross-region replication of the stored message -- async
```

Group fan-out specifically: for a 5,000-member Slack channel, don't do 5,000 synchronous pushes inline on the sender's request. Enqueue the fan-out (Kafka topic keyed by `conversation_id`, or a dedicated fan-out worker pool) so one slow or offline member's lookup doesn't add latency to the other 4,999, and so a burst of large-group messages doesn't back up the gateway that received the send.

11. Caching

Interviewer: Where does a cache help here?

Candidate:

- **Session/presence is already effectively a cache** — Redis holding `userId -> gatewayId` and `online/last_seen` is the fast path; there's no slower backing store behind it that needs invalidation, because this data is soft-state anyway.
 - **Recent messages per active conversation** — cache the last ~50 messages per open conversation (Redis or an in-gateway LRU) so reopening a chat or syncing a second device doesn't hit the message store for the common "just show me the recent thread" case. Cache-aside: read cache → miss → read store → populate cache.
 - **Cold scroll-back is never cached** — a user paging to messages from 8 months ago is rare and low-value to keep warm; it goes straight to the message store's range scan.
 - **Invalidation is naturally time-ordered, not "invalidate on write" complexity** — a new message just appends to the cached recent-window (push in, evict oldest), no stale-read risk the way a mutable-record cache would have.
-

12. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Real-time channel (client ↔ gateway)	WebSocket	One persistent, full-duplex TCP connection — server <i>pushes</i> the instant a message arrives, no per-message handshake. Not polling: short-polling means constant "anything new?" requests (wasteful, plus polling-interval latency); long-polling still pays a reconnect cost every cycle and isn't true duplex push.
Connection registry (userId → gateway)	Redis	Sub-millisecond in-memory lookup, naturally shardable by <code>userId</code> , TTL gives free stale-session detection on ungraceful gateway death. Not a SQL table: rewritten on every heartbeat — that write rate would hammer a durable store for data we're fine losing on restart.
Message store	Cassandra (or DynamoDB)	Write-heavy (700k-900k/s peak), access pattern is "all messages for one <code>conversation_id</code> , time-ordered" — maps directly onto partition-by-conversation, cluster-by-time-based-ID, giving O(1) partition reads sorted newest-first, and horizontal write scaling by adding nodes. Not a single relational instance: can't sustain that write volume, and we don't need cross-conversation joins or multi-row transactions — just fast appends and range scans.
Internal cross-gateway fan-out (1:1)	Direct gRPC call, address from registry	Sender's gateway already knows the exact target gateway; a direct call is the lowest-latency path for a single recipient.
Internal fan-out (groups)	Pub/sub (Redis Pub/Sub or Kafka topic per conversation)	One publish reaches every gateway holding a member's socket, without the sender's gateway needing to enumerate thousands of individual targets. Not thousands of direct gRPC calls per send: doesn't scale with group size and couples the sender's request latency to the slowest recipient's gateway.
Offline delivery	Store-and-forward + push notification (APNs/FCM)	The durable write already happened regardless of recipient state; a missing registry entry just triggers a notification instead of a live push — no separate "pending message" subsystem needed.
Presence	Redis, short TTL, heartbeat-refreshed	Presence is small, volatile, constantly overwritten — perfect for in-memory KV with expiry; a durable-store write per heartbeat would be wasted I/O for data nobody needs after a few seconds.
Load balancer	L7 for the WebSocket upgrade, pinned/passthrough after	Only L7 can see the <code>Upgrade: websocket</code> header to route it correctly; after the handshake the connection is sticky to one gateway for its lifetime — there's nothing left to "load balance" per request.
Media (images/video)	Object storage (S3) + CDN, chat store keeps only a reference	Media is large and immutable once uploaded — CDN-fronted blob storage is built for exactly that; the message row just

Concern	Choice	Why this, and why not the alternative
		holds a URL, keeping the hot write path (Cassandra) small and fast.
Idempotent sends	Client-generated message ID, INSERT IF NOT EXISTS	The client is the only party that knows "this is a retry of what I already tried" — a flaky connection causing a resend must not create a visible duplicate.

The one-sentence justification you'd say out loud: *"WebSockets give the server a live push channel to every client; a sharded Redis registry maps user to the exact gateway holding their socket so any server can route to any user; every message is durably written to a partition-by-conversation, time-clustered Cassandra store before it's ever pushed, so a dropped socket never loses data and offline users just get a push notification instead; and receipts, fan-out, and replication all happen asynchronously so none of them sit on the sub-200ms delivery path."*

13. Failure modes & graceful degradation

Failure	What happens	Mitigation
Gateway server crashes	All sockets it held drop instantly	Clients detect via missed heartbeat, reconnect to a new gateway (via LB), re-register session in Redis; in-flight messages already persisted are unaffected
Registry entry goes stale (gateway died without clean disconnect)	Lookups for that user briefly return "offline" incorrectly	TTL bounds the blast radius to a few seconds; heartbeat on reconnect refreshes it
Redis registry shard down	Lookups for its slice of users fail	Redis replicas + failover; worst case those messages fall back to store-and-forward + push notification rather than live delivery — degraded, not lost
Cassandra node down	—	RF=3 with quorum reads/writes keeps serving; hinted handoff + repair heal it in the background
Cross-region link down	Cross-region messages can't push live	Local delivery unaffected; messages queue at the edge and flush once the link heals; already persisted locally so nothing is lost
Push notification provider (APNs/FCM) down	Offline users don't get a phone banner	Message is still durably stored and appears the next time they open the app; not on the durability path, only the "wake them up" path
Massive group send (5,000 members)	Naive fan-out could stall the sender's gateway	Async fan-out queue decouples send latency from recipient count

14. Follow-up curveballs

Interviewer: How do read receipts work in a 5,000-person channel without writing a row per member per message?

Candidate: Don't track per-message-per-reader state at all — track a **last-read pointer** per user per conversation (`user_id, conversation_id -> last_read_message_id`). "Unread count" is just "messages after that pointer" computed on read, and "who's read this" for small groups can scan pointers $\geq$ the message's ID; for huge channels we typically drop per-message read receipts entirely (Slack does this) and just show unread counts, because per-reader-per-message state genuinely doesn't scale past a certain group size.

Interviewer: A user has three devices open — phone, laptop, tablet. How does that change the session registry and delivery?

Candidate: The registry maps `userId -> a set of (gatewayId, deviceId)` pairs, not a single value. A send fans out to every live session for that user, and each device independently tracks its own delivered/read state so read-on-laptop can (optionally) also mark it read on phone — that's a small sync message broadcast to the user's other devices, same fan-out mechanism as a group send of size N-1.

Interviewer: What about end-to-end encryption — does any of this design change?

Candidate: Barely, at this altitude. The server routes and stores **ciphertext blobs** it cannot read; ordering, persistence, receipts, and routing all operate on opaque payloads exactly as before. What changes is entirely client-side: key exchange (e.g. a Signal-protocol-style ratchet) happens between clients, and the server's only involvement is storing/relaying public-key bundles for initial handshakes — it never holds a decryption key. Group encryption adds complexity (rotating a group key on membership change) but that's a client-side crypto concern, not a backend routing/storage one.

Interviewer: How would you detect and handle "typing..." indicators without flooding the system?

Candidate: Typing indicators are the most disposable data in the system — never persist them, never route them through the durable store. Send them over the same WebSocket as an ephemeral event, fan out only to currently-open sessions in that conversation (same registry lookup, skip persistence entirely), and debounce on the client (send at most once every couple seconds, plus a "stopped typing" after a timeout) so a fast typist doesn't spam an event per keystroke.

15. Deep-dive: "Why not just poll, or long-poll, instead of holding WebSockets open?"

Interviewer: A hundred million idle open sockets sounds expensive and fragile. Why not have the client just **ask** every few seconds — GET /messages?since=<cursor> — or use long-polling? Simpler infra, no gateway fleet holding state.

Candidate: It's the right instinct to question — "hold millions of idle connections open" is an unusual amount of standing infrastructure to justify, so let me be precise about why plain polling loses, why long-polling is closer but still loses for *this* problem, and where that instinct is actually correct for a *different* problem.

Short polling — pay latency AND server load, simultaneously

```
Client:  GET /messages?since=T  →  Server: "nothing new"      (repeat every N seconds)
        GET /messages?since=T  →  Server: "nothing new"
        GET /messages?since=T  →  Server: "msg#42!"          <-- arrived between polls,
                                                                sits unseen until next t
```

The failure is two-sided, not one-sided: - **Latency is capped by the poll interval.** Poll every 3s and the average delay to notice a new message is ~1.5s, worst case 3s — nowhere near the <200ms bar we set. - **Server load is dominated by empty answers.** At 100M users polling every 3s, that's ~33M requests/sec hitting the backend, and the overwhelming majority say "nothing new" — we'd be paying full request-handling cost (auth, routing, a DB/cache read) for a 204 almost every time. Tightening the interval to cut latency makes this worse in lockstep — there is no dial that fixes both problems at once, because they trade directly against each other.

Long-polling — closes the latency gap, keeps the connection-count problem

```
Client:  GET /messages?since=T  →  Server holds the request open,
                                   does NOT respond until either
                                   (a) a message arrives -> respond immediately, or
                                   (b) a timeout (~30-60s) -> respond empty, client re-pol
```

This is a real improvement — the server pushes the instant it has something, so latency looks like WebSocket latency while a message is in flight. But look at what it costs: the server is still holding **one open HTTP request per connected client**, for up to a minute at a time, before the client tears it down and immediately re-opens another one. That's the *same* "100M things held open" problem as WebSocket, except: - each cycle pays a **fresh HTTP request/response handshake** (headers, routing, sometimes a new TCP/TLS setup) roughly once a minute per client, at 100M clients — that's real recurring connection-churn overhead a WebSocket never pays after its one handshake. - it's still **half-duplex** — the client can't push (typing indicators, presence) on the same channel; you'd need a second channel for that, which WebSocket gives you for free in one connection.

Why holding idle sockets is still the *cheaper* idle cost

The core rebuttal to "isn't holding 100M sockets expensive?" — yes, but everything else pays an idle cost too, and WebSocket's is the smallest per-connection:

Connection model	New request every interval	Request held open, re-issued on timeout/response	One long-lived HTTP response stream	One long-lived, full-duplex socket
Typical delivery latency	~poll interval / 2 (avg)	near-instant while held	near-instant	near-instant
Server idle cost per client	none between polls, but huge request <i>volume</i>	one held request + periodic re-handshake churn	one held response, one-way	one socket, both directions
Duplex (client can also push live)?	no (separate request)	no (separate request)	no — server → client only	yes, native
Reconnect/handshake overhead	one per poll (frequent, cheap each)	one per timeout cycle (~1/min, at scale = huge aggregate churn)	one, rare	one, rare
Works through corporate proxies/older infra	trivially (plain HTTP)	mostly	mostly (plain HTTP streaming)	sometimes blocked; needs fallback
Good fit for	rare/low-value updates (a badge count)	simple notify-style updates, infra that can't do WS	server→client feeds (live scores, stock tickers)	true bidirectional real-time (chat, typing, presence)

When long-polling/SSE is actually the right call: if the channel is **one-directional** (server tells the client things; the client never needs to push back on the same channel) and connection counts are modest or infra constraints (some enterprise proxies, older load balancers) make WebSocket awkward — SSE is a genuinely good, simpler choice; it's a plain HTTP response the browser/CDN stack already understands, with automatic reconnect built into the `EventSource` spec. That's why a stock-ticker or a live-score widget often reaches for SSE instead of WebSocket. Chat fails that test on one axis: the client must *also* push (sending messages, typing indicators) at the same real-time cadence it receives — so we'd need a second channel for the upload direction anyway, at which point we've built two half-duplex channels to avoid one full-duplex one, for no savings.

Rule of thumb: *Polling* trades latency for simplicity and only makes sense for low-value, infrequent updates. *Long-polling* buys back latency but still churns a connection per cycle and stays half-duplex. *SSE* is the right choice for pure server→client feeds. *WebSocket* is the only one of the four that gives true low-latency duplex — pay the idle-connection cost once per client for the life of the session, instead of paying a request/handshake cost repeatedly, forever, for every client, every few seconds.

16. Deep-dive: recipient reconnects to a different gateway — ordering + exactly-once, walked through

Interviewer: Here's the sharpest edge case in your design. User B is connected to Gateway A. They go offline — maybe just a subway tunnel, a few seconds — and while they're gone, someone sends them a message. B comes back, but this time they land on **Gateway B**, a completely different physical machine that shares no memory with Gateway A. Walk me through, in detail, how that message finds B, arrives **in order**, and arrives **exactly once** — not zero times, not twice.

Candidate: This is exactly the scenario the registry + persist-before-push design was built for, but it's worth walking through the failure mode precisely, because the naive version of "just forward it" breaks in a very specific way: a **race between the forward and the reconnect**.

The setup

```
t0: Client B is connected to GATEWAY A.  
    Session registry: session:B = { gateway: A, deviceId: d1 }  
  
t1: Sender A' sends msg to conversation C, addressed to B.  
    Gateway X (A's own gateway) does:  
    1. persist to durable store: (conversation=C, seq=41, body, sender=A')  
    2. registry lookup: session:B -> Gateway A  
    3. forward "seq 41" over internal link to Gateway A  
    4. Gateway A tries to push over B's socket → x socket already dead  
                                           (B's subway dip happened  
                                           between step 2 and step 4)
```

The race, made concrete

```
t2: B's client detects the drop, reconnects -- lands on GATEWAY B this time  
    (different machine, different AZ even). Registry updated:  
    session:B = { gateway: B, deviceId: d1 } <-- overwrites the stale entry  
  
t3: Sender A' sends a SECOND message, seq=42, moments later.  
    Gateway X looks up the registry NOW -> gets Gateway B (fresh) -> forwards ->  
    Gateway B pushes it straight to B's live socket -> B sees "seq 42" IMMEDIATELY.
```

Result if we do nothing else:

B has rendered seq 42, and seq 41 — sent FIRST — is nowhere on B's screen.
That's simultaneously a LOST message (until something rescues it) and an
OUT-OF-ORDER render (42 before 41) the instant it does show up.

The registry did its job — it always returns the *current* gateway. The bug isn't in the registry; it's that a **live forward is a best-effort, fire-and-forget hop**, and nothing observed that hop 4 (the actual push) failed. Gateway A had no way to tell Gateway X "I received your forward for B but couldn't deliver it" in this simple version, and even if it did, by the time that news traveled back, B might have already reconnected elsewhere.

What must be true for correctness

1. **The message must never depend on the forward succeeding.** It's already durably persisted at t1 step 1, before we know anything about B's socket — that part of the design already holds.
Good: nothing is *permanently* lost. It's just not *yet delivered*.
2. **B's client must be able to detect it has a gap,** not just trust whatever the live socket handed it.
This is the missing piece above — B rendered 42 with no way to know 41 existed.
3. **Redelivery of 41 must not create a duplicate if it *also* arrives live from some in-flight retry.**
Store-and-forward and live push can both eventually try to deliver the same message.

Ranked fixes

Fix 1 — blind retry/redelivery on forward failure (naive). Gateway A notices the push failed and reports back to Gateway X, which retries the forward... but to where? It has to re-check the registry (now points at Gateway B), so it becomes a second forward attempt to a second gateway. This *can* work but only if every intermediate hop reliably detects failure and retries with a fresh lookup — fragile, and says nothing about *ordering* relative to seq 42 which may have already been delivered live in the meantime. Doesn't solve the ordering half of the problem at all.

Fix 2 — "flush the offline queue" on reconnect, no sequencing. When B reconnects, Gateway B asks the store for "everything undelivered for B" and pushes it. Better — it guarantees 41 eventually arrives — but with no ordering key, nothing stops it from arriving *after* 42 was already rendered (exactly what we saw), and nothing stops it from being pushed twice if a retry from Fix 1 is also in flight. Solves loss, not order, not duplication.

Fix 3 — per-conversation monotonic sequence number + client resume protocol (recommended). Give every message in a conversation a **strictly increasing per-conversation sequence number**, assigned atomically at persist time (a Redis `INCR conv:{id}:seq`, or an equivalent atomic counter/lightweight transaction in the store) — *not* a wall-clock timestamp, for the same clock-skew reason as §9. Each client tracks `last_applied_seq` per conversation. The protocol on every (re)connect, before trusting any live push:

```
B reconnects to GATEWAY B
|
| 1. write registry: session:B = {gateway: B}           (routing fixed immediately)
|
| 2. client tells server: "resume conversation C from seq=40"
|    (40 = last seq B actually rendered, persisted client-side)
|
| 3. server reads store: SELECT seq, body WHERE conversation=C AND seq > 40
|    -> returns [41, 42] IN ORDER, from the single source of truth
|
| 4. client applies 41, then 42, in that order, updating last_applied_seq as it goes
|
| 5. ONLY NOW does the client start trusting live pushes for C, and even
|    those are checked: incoming seq == last_applied_seq + 1 ? apply it
|                        incoming seq > last_applied_seq + 1 ? there's a
|                          gap -> re-run step 3 to fill it, don't render yet
|                        incoming seq <= last_applied_seq ? DUPLICATE, drop it
```

That last line is the exactly-once guarantee: **the client de-dupes by sequence number (or the client-generated message ID from §12's idempotent-send rule), not by trusting "I only got it once."** It's fine — expected, even — for the same message to arrive via both a stale retried forward *and* the catch-

up read; the client's job is to apply each seq exactly once and ignore repeats. This is the same idempotency principle as the message-store's `INSERT IF NOT EXISTS` on the write side, mirrored on the read/render side.

Fix 4 — server-side delivery acks to suppress redundant redelivery (complementary, not a replacement). Once B's client acks "received seq 41," the server can stop retrying that message. This trims wasted redelivery traffic but is an optimization, not a correctness mechanism — the client-side seq check in Fix 3 is what actually guarantees exactly-once *render*, because acks themselves can be lost too.

Recommended combination: persist-before-push (already in the base design) for never-truly-lost, **plus** a per-conversation atomic sequence number assigned at write time, **plus** a client resume protocol that always reads "since my last seq" on every reconnect before trusting the live channel, **plus** client-side dedup keyed on that sequence number (or message ID). The registry (Redis `userId -> gateway`) still does exactly one job in this picture — routing the *next* live push to the *current* socket — and nothing more; it was never supposed to be the thing guaranteeing order or exactly-once, and trying to make it do that is where the naive version goes wrong.

What to say out loud: *"The registry only answers 'where do I route this right now' — it's not the durability or ordering mechanism, so I don't lean on it for that. Durability comes from persisting before any push is attempted. Ordering comes from a per-conversation monotonic sequence number assigned at persist time, not a timestamp. And exactly-once comes from the client always resuming from its last-applied sequence number on reconnect — reading the gap from the store before trusting live pushes — and deduping anything it's already applied. Three different jobs, three different mechanisms; that's why a single 'just forward it again' fix can't cover all of them."*

Summary — the mental model

Chat is fundamentally a **push, not a pull, problem**: the hard part isn't the database, it's that hundreds of millions of clients hold an open socket and the server must find and use the *right* one, on the *right* machine, in under 200ms. The winning shape: a **fleet of stateful gateway servers** each holding a slice of sockets, a **sharded Redis registry** mapping user to gateway so any server can route to any user, and a **write-heavy, partition-by-conversation message store (Cassandra)** that is always the durable source of truth *before* anything is pushed live — so a dropped connection degrades to store-and-forward plus a push notification, never to data loss. Ordering rides on time-sortable IDs rather than wall clocks; receipts are a monotonic state machine; and everything that isn't "get the message on screen" — fan-out beyond the first hop, notifications, replication, receipts propagation — runs asynchronously off the critical path.